OOP, STL & Modern C++

Complete Mastery with 4 New Projects

Deep Concepts • Real Code • Production Ready

Professional Development Guide

March 6, 2026

Contents

Part I

OOP Deep Dive — The 4 Pillars

Chapter 1

Encapsulation — Data Hiding Mastery

Definition

Encapsulation = Data (attributes) aur methods (functions) ko ek unit (class) mein band karna, aur external access ko control karna through **access specifiers** (private, protected, public).

1.1 Access Specifiers Deep Dive

Listing 1.1: encapsulation_demo.hpp — Complete Access Control

```
#ifndef ENCAPSULATION_DEMO_HPP
#define ENCAPSULATION_DEMO_HPP

#include <string>
#include <stdexcept>
#include <iostream>

class SecureBankAccount {
    // PRIVATE: Sirf class ke andar access
    std::string accountNumber;
    double balance;
    std::string passwordHash;
    int failedLoginAttempts;

    // PROTECTED: Class + Derived classes
protected:
    std::string accountType;
    double interestRate;

    // PUBLIC: Sab access kar sakte hain
public:
    std::string ownerName;  // Generally avoid public data members

    // Constructor with validation
```

```
    SecureBankAccount(std::string accNum, std::string owner,
                      std::string pwd, double initialBalance);

    // DESTRUCTOR: Cleanup (RAII)
    ~SecureBankAccount();

    // GETTERS: Controlled read access
    double getBalance() const;            // const = doesn't modify
    std::string getAccountNumber() const;
    std::string getMaskedAccountNumber() const;  // Security: last 4 digits only

    // SETTERS: Controlled write access with validation
    void deposit(double amount);
    bool withdraw(double amount, const std::string& password);
    void setPassword(const std::string& oldPwd, const std::string& newPwd);

    // Business logic methods
    void applyInterest();
    void freezeAccount();  // After 3 failed attempts

private:
    // PRIVATE METHODS: Internal helpers
    bool verifyPassword(const std::string& pwd) const;
    void logTransaction(const std::string& type, double amount);
    std::string hashPassword(const std::string& pwd) const;
};

// INLINE DEFINITIONS (for small functions)
inline double SecureBankAccount::getBalance() const {
    return balance;
}

#endif
```

Listing 1.2: encapsulation_demo.cpp — Implementation

```
#include "encapsulation_demo.hpp"
#include <iomanip>
#include <sstream>

SecureBankAccount::SecureBankAccount(std::string accNum, std::string owner,
                                     std::string pwd, double initialBalance)
    : accountNumber(accNum), ownerName(owner),
      passwordHash(hashPassword(pwd)), balance(initialBalance),
      failedLoginAttempts(0), accountType("Savings"), interestRate(0.04) {
```

```

    if (initialBalance < 0) {
        throw std::invalid_argument("Initial balance cannot be negative");
    }
    if (accNum.length() != 10) {
        throw std::invalid_argument("Account number must be 10 digits");
    }
}

SecureBankAccount::~SecureBankAccount() {
    // Log account closure, cleanup resources
    std::cout << "Account " << getMaskedAccountNumber() << " closed\n";
}

std::string SecureBankAccount::getMaskedAccountNumber() const {
    return "XXXX-XXXX-" + accountNumber.substr(6, 4);
}

void SecureBankAccount::deposit(double amount) {
    if (amount <= 0) {
        throw std::invalid_argument("Deposit amount must be positive");
    }
    balance += amount;
    logTransaction("DEPOSIT", amount);
}

bool SecureBankAccount::withdraw(double amount, const std::string& password) {
    if (failedLoginAttempts >= 3) {
        throw std::runtime_error("Account frozen due to failed attempts");
    }

    if (!verifyPassword(password)) {
        failedLoginAttempts++;
        return false;
    }

    failedLoginAttempts = 0;  // Reset on success

    if (amount > balance) {
        throw std::runtime_error("Insufficient funds");
    }

    balance -= amount;
```

```
    logTransaction("WITHDRAW", amount);
    return true;
}

bool SecureBankAccount::verifyPassword(const std::string& pwd) const {
    return hashPassword(pwd) == passwordHash;
}

std::string SecureBankAccount::hashPassword(const std::string& pwd) const {
    // Simple hash (production mein bcrypt/scrypt use karo)
    std::hash<std::string> hasher;
    return std::to_string(hasher(pwd));
}

void SecureBankAccount::logTransaction(const std::string& type, double amount) {
    std::cout << "[" << type << "] " << std::fixed << std::setprecision(2)
              << amount << " | Balance: " << balance << "\n";
}
```

Encapsulation Best Practices

1. **Always private by default:** Data members hamesha private rakho
2. **Getters for read-only:** const reference return karo large objects ke liye
3. **Setters with validation:** Har modification pe check karo
4. **Immutability:** Agar change nahi hota, toh const methods banao
5. **Friend classes:** Sirf zaroorat mein use karo (tight coupling)

Chapter 2

Inheritance — Code Reuse Hierarchy

Definition

Inheritance = Existing class (Base/Parent) ki properties aur behaviors ko nayi class (Derived/Child) mein acquire karna. `is-a` relationship establish karta hai.

2.1 Types of Inheritance

Listing 2.1: inheritance_types.hpp — All 5 Types

```
#ifndef INHERITANCE_TYPES_HPP
#define INHERITANCE_TYPES_HPP

#include <iostream>
#include <string>

//
// BASE CLASS: Person
//

class Person {
protected:
    std::string name;
    int age;
    std::string id;

public:
    Person(std::string n, int a, std::string i)
        : name(n), age(a), id(i) {
        std::cout << "Person constructed: " << name << "\n";
    }

```

```
    virtual ~Person() {
        std::cout << "Person destroyed: " << name << "\n";
    }

    virtual void display() const {
        std::cout << "[Person] " << name << ", " << age << " years\n";
    }

    virtual double calculateBenefits() const = 0;  // Pure virtual
};

//


// 1. SINGLE INHERITANCE: Student is-a Person
//


class Student : public Person {
private:
    std::string studentId;
    std::string major;
    double gpa;
    std::vector<std::string> courses;

public:
    Student(std::string n, int a, std::string i,
            std::string sid, std::string m, double g)
        : Person(n, a, i), studentId(sid), major(m), gpa(g) {}

    void enrollCourse(const std::string& course) {
        courses.push_back(course);
    }

    void display() const override {
        std::cout << "[Student] " << name << " | ID: " << studentId
                  << " | Major: " << major << " | GPA: " << gpa << "\n";
    }

    double calculateBenefits() const override {
        return gpa > 3.5 ? 5000.0 : 0.0;  // Scholarship
    }
};

//
```

```
// 2. MULTILEVEL INHERITANCE: GraduateStudent is-a Student is-a Person
//


class GraduateStudent : public Student {
private:
    std::string researchTopic;
    std::string advisor;
    bool isTeachingAssistant;

public:
    GraduateStudent(std::string n, int a, std::string i,
                    std::string sid, std::string m, double g,
                    std::string topic, std::string adv, bool ta)
        : Student(n, a, i, sid, m, g),
          researchTopic(topic), advisor(adv), isTeachingAssistant(ta) {}

    void display() const override {
        Student::display();  // Base class display
        std::cout << "  [Grad] Research: " << researchTopic
                  << " | Advisor: " << advisor << "\n";
    }

    double calculateBenefits() const override {
        double base = Student::calculateBenefits();
        if (isTeachingAssistant) base += 15000;  // Stipend
        return base;
    }
};

//


// 3. HIERARCHICAL INHERITANCE: Professor & Student both inherit Person
//


class Professor : public Person {
private:
    std::string department;
    std::string rank;  // Assistant, Associate, Full
    double salary;
    std::vector<std::string> publications;

public:
    Professor(std::string n, int a, std::string i,
```

```
                std::string dept, std::string r, double sal)
        : Person(n, a, i), department(dept), rank(r), salary(sal) {}

    void addPublication(const std::string& pub) {
        publications.push_back(pub);
    }

    void display() const override {
        std::cout << "[Professor] " << name << " | " << rank
                  << " Prof, " << department << "\n";
    }

    double calculateBenefits() const override {
        double bonus = publications.size() * 1000;
        if (rank == "Full") bonus += 20000;
        return bonus;
    }
};

//


// 4. MULTIPLE INHERITANCE: TeachingAssistant is-a Student AND Employee
//         Diamond Problem ka dhyan rakhna!
//


class Employee {
protected:
    std::string employeeId;
    double baseSalary;
    int yearsOfService;

public:
    Employee(std::string eid, double sal, int years)
        : employeeId(eid), baseSalary(sal), yearsOfService(years) {}

    virtual double calculateSalary() const {
        return baseSalary + (yearsOfService * 1000);
    }
};

// Virtual inheritance to solve Diamond Problem
class TeachingAssistant : public Student, public Employee {
private:
    std::string assignedCourse;
```

```
    int hoursPerWeek;

public:
    TeachingAssistant(std::string n, int a, std::string i,
                      std::string sid, std::string m, double g,
                      std::string eid, double sal, int years,
                      std::string course, int hours)
        : Person(n, a, i),  // Virtual base called explicitly
          Student(n, a, i, sid, m, g),
          Employee(eid, sal, years),
          assignedCourse(course), hoursPerWeek(hours) {}

    void display() const override {
        std::cout << "[TA] " << name << " | StudentID: " << id
                  << " | EmpID: " << employeeId
                  << " | Course: " << assignedCourse << "\n";
    }

    double calculateBenefits() const override {
        return Student::calculateBenefits() +
               (hoursPerWeek * 50);  // Hourly pay
    }
};

//


// 5. HYBRID INHERITANCE: Complex hierarchy combining above
//


class ResearchAssistant : public GraduateStudent, public Employee {
    std::string labName;
    std::string grantId;

public:
    ResearchAssistant(std::string n, int a, std::string i,
                      std::string sid, std::string m, double g,
                      std::string topic, std::string adv, bool ta,
                      std::string eid, double sal, int years,
                      std::string lab, std::string grant)
        : Person(n, a, i),
          GraduateStudent(n, a, i, sid, m, g, topic, adv, ta),
          Employee(eid, sal, years),
          labName(lab), grantId(grant) {}

    double calculateBenefits() const override {
        return GraduateStudent::calculateBenefits() +
```

```
                Employee::calculateSalary() * 0.1;  // 10% research bonus
    }
};

#endif
```

Diamond Problem Virtual Inheritance

Problem: Agar `class D : public B, public C` aur `B` aur `C` dono `class A` se inherit karte hain, toh `D` mein `A` ke 2 copies ban jate hain.
Solution: Virtual Inheritance

```
class B : virtual public A { ... };
class C : virtual public A { ... };
class D : public B, public C {
    D() : A(...), B(...), C(...) {}  // A explicitly call karna padta hai
};
```

Chapter 3

Polymorphism — Many Forms

Definition

Polymorphism = Same interface, different implementations. 2 types:

- **Compile-time (Static):** Function overloading, Operator overloading, Templates
- **Run-time (Dynamic):** Virtual functions, Abstract classes

3.1 Complete Polymorphism Example

Listing 3.1: polymorphism_complete.hpp — Runtime Polymorphism

```cpp
#ifndef POLYMORPHISM_COMPLETE_HPP
#define POLYMORPHISM_COMPLETE_HPP

#include <iostream>
#include <vector>
#include <memory>
#include <cmath>

//
// ABSTRACT BASE CLASS: Pure Interface
//
class Shape {
public:
    virtual ~Shape() = default;

    // Pure virtual functions = MUST implement in derived
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
```

```
    virtual void draw() const = 0;
    virtual void scale(double factor) = 0;

    // Virtual with default implementation
    virtual void printInfo() const {
        std::cout << "Area: " << area()
                  << ", Perimeter: " << perimeter() << "\n";
    }

    // Non-virtual: Common to all shapes
    std::string getType() const { return "Shape"; }
};

//


// CONCRETE SHAPES
//



class Circle : public Shape {
private:
    double radius;
    mutable int areaCacheValid;  // mutable: can change in const
       method
    mutable double cachedArea;

public:
    explicit Circle(double r) : radius(r), areaCacheValid(false) {
        if (r <= 0) throw std::invalid_argument("Radius must be
           positive");
    }

    double area() const override {
        if (!areaCacheValid) {
            cachedArea = M_PI * radius * radius;
            areaCacheValid = true;
        }
        return cachedArea;
    }

    double perimeter() const override {
        return 2 * M_PI * radius;
    }

    void draw() const override {
        std::cout << "Drawing Circle (r=" << radius << ")\n";
    }
```

```

    void scale(double factor) override {
        radius *= factor;
        areaCacheValid = false;  // Invalidate cache
    }

    double getRadius() const { return radius; }
};

class Rectangle : public Shape {
private:
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h) {
        if (w <= 0 || h <= 0)
            throw std::invalid_argument("Dimensions must be positive");
    }

    double area() const override { return width * height; }
    double perimeter() const override { return 2 * (width + height); }

    void draw() const override {
        std::cout << "Drawing Rectangle (" << width << "x" << height << ")\n";
    }

    void scale(double factor) override {
        width *= factor;
        height *= factor;
    }

    bool isSquare() const { return width == height; }
};

class Triangle : public Shape {
private:
    double a, b, c;  // Three sides

public:
    Triangle(double side1, double side2, double side3)
        : a(side1), b(side2), c(side3) {
        // Triangle inequality check
        if (a + b <= c || a + c <= b || b + c <= a)
            throw std::invalid_argument("Invalid triangle sides");
    }

```

```
    double area() const override {
        // Heron's formula
        double s = (a + b + c) / 2;
        return sqrt(s * (s - a) * (s - b) * (s - c));
    }

    double perimeter() const override { return a + b + c; }

    void draw() const override {
        std::cout << "Drawing Triangle (sides: " << a << "," << b << "," << c << ")\n";
    }

    void scale(double factor) override {
        a *= factor; b *= factor; c *= factor;
    }

    std::string getType() const {
        if (a == b && b == c) return "Equilateral";
        if (a == b || b == c || a == c) return "Isosceles";
        return "Scalene";
    }
};

//


// POLYMORPHIC CONTAINER & FACTORY
//


class ShapeFactory {
public:
    static std::unique_ptr<Shape> createCircle(double radius) {
        return std::make_unique<Circle>(radius);
    }

    static std::unique_ptr<Shape> createRectangle(double w, double h) {
        return std::make_unique<Rectangle>(w, h);
    }

    static std::unique_ptr<Shape> createTriangle(double a, double b, double c) {
        return std::make_unique<Triangle>(a, b, c);
    }
};

class ShapeManager {
```

```
private:
    std::vector<std::unique_ptr<Shape>> shapes;

public:
    void addShape(std::unique_ptr<Shape> shape) {
        shapes.push_back(std::move(shape));
    }

    void drawAll() const {
        std::cout << "=== Drawing All Shapes ===\n";
        for (const auto& shape : shapes) {
            shape->draw();  // Polymorphic call!
        }
    }

    double totalArea() const {
        double total = 0;
        for (const auto& shape : shapes) {
            total += shape->area();  // Runtime dispatch
        }
        return total;
    }

    void scaleAll(double factor) {
        for (auto& shape : shapes) {
            shape->scale(factor);
        }
    }

    void printAllInfo() const {
        for (const auto& shape : shapes) {
            shape->printInfo();
        }
    }
};

#endif
```

Listing 3.2: polymorphism_demo.cpp — Runtime Polymorphism in Action

```
#include "polymorphism_complete.hpp"

int main() {
    ShapeManager manager;

    // Factory pattern se shapes create karo
    manager.addShape(ShapeFactory::createCircle(5.0));
    manager.addShape(ShapeFactory::createRectangle(4.0, 6.0));
    manager.addShape(ShapeFactory::createTriangle(3.0, 4.0, 5.0));

```

```
    // Polymorphic behavior
    manager.drawAll();
    manager.printAllInfo();

    std::cout << "\nTotal Area: " << manager.totalArea() << "\n";

    // Scale all shapes uniformly
    manager.scaleAll(2.0);
    std::cout << "\nAfter scaling 2x:\n";
    manager.printAllInfo();

    return 0;
}
```

```
g++ -std=c++17 -Wall -o poly polymorphism_demo.cpp && ./poly
=== Drawing All Shapes ===
Drawing Circle (r=5)
Drawing Rectangle (4x6)
Drawing Triangle (sides: 3,4,5)
Area: 78.5398, Perimeter: 31.4159
...
```

Virtual Function Internals — Vtable

Kaise kaam karta hai?

1. Har class mein **vtable** (virtual table) hoti hai — function pointers ka array
2. Har object mein **vptr** (virtual pointer) hota hai — vtable ko point karta hai
3. `shape->area()` call pe: vptr → vtable → actual function

Cost: Extra memory (vptr per object) + indirect function call. But flexibility worth it!

Chapter 4

Abstraction — Hiding Complexity

Definition

Abstraction = Complex implementation ko hide karke simple interface provide karna. **Abstract classes** (pure virtual functions) aur **interfaces** se achieve hota hai.

Listing 4.1: abstraction_complete.hpp — Interface Design

```
#ifndef ABSTRACTION_COMPLETE_HPP
#define ABSTRACTION_COMPLETE_HPP

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <chrono>

//
// PURE INTERFACE: Database Connection
//
class IDatabaseConnection {
public:
    virtual ~IDatabaseConnection() = default;

    // Connection management
    virtual bool connect(const std::string& connectionString) = 0;
    virtual void disconnect() = 0;
    virtual bool isConnected() const = 0;

    // Transaction support
    virtual void beginTransaction() = 0;
    virtual void commit() = 0;
```

```
    virtual void rollback() = 0;

    // Query execution
    virtual std::vector<std::vector<std::string>>
        executeQuery(const std::string& sql) = 0;
    virtual int executeUpdate(const std::string& sql) = 0;

    // Metadata
    virtual std::string getLastError() const = 0;
    virtual std::string getServerVersion() const = 0;
};

//


// CONCRETE IMPLEMENTATIONS: Different Databases
//



class MySQLConnection : public IDatabaseConnection {
private:
    std::string host;
    int port;
    bool connected;
    std::string lastError;
    // ... actual MySQL connection handle ...

public:
    MySQLConnection(std::string h, int p)
        : host(h), port(p), connected(false) {}

    bool connect(const std::string& connStr) override {
        std::cout << "[MySQL] Connecting to " << host << ":" <<
           port << "\n";
        // Actual MySQL connection logic
        connected = true;
        return true;
    }

    void disconnect() override {
        std::cout << "[MySQL] Disconnecting\n";
        connected = false;
    }

    bool isConnected() const override { return connected; }

    void beginTransaction() override {
        executeUpdate("START TRANSACTION");
```

```
    }

    void commit() override {
        executeUpdate("COMMIT");
    }

    void rollback() override {
        executeUpdate("ROLLBACK");
    }

    std::vector<std::vector<std::string>>
    executeQuery(const std::string& sql) override {
        std::cout << "[MySQL] Executing: " << sql << "\n";
        // Return mock data
        return {{"1", "Rahul", "25"}, {"2", "Priya", "28"}};
    }

    int executeUpdate(const std::string& sql) override {
        std::cout << "[MySQL] Updating: " << sql << "\n";
        return 1;  // rows affected
    }

    std::string getLastError() const override { return lastError; }
    std::string getServerVersion() const override { return "8.0.32"
        ; }
};

class PostgreSQLConnection : public IDatabaseConnection {
    // Similar implementation for PostgreSQL
    // Same interface, different internals
};

class SQLiteConnection : public IDatabaseConnection {
    // For embedded/local database
};

//


// HIGH-LEVEL CLASS: Uses abstraction, doesn't care about
    implementation
//


class UserRepository {
private:
    std::unique_ptr<IDatabaseConnection> db;

public:
```

```
    explicit UserRepository(std::unique_ptr<IDatabaseConnection> connection)
        : db(std::move(connection)) {}

    bool addUser(const std::string& name, int age) {
        if (!db->isConnected() && !db->connect("default")) {
            return false;
        }

        db->beginTransaction();
        try {
            std::string sql = "INSERT INTO users (name, age) VALUES ('"
                              + name + "', " + std::to_string(age) + ")";
            int result = db->executeUpdate(sql);
            db->commit();
            return result > 0;
        } catch (...) {
            db->rollback();
            return false;
        }
    }

    std::vector<std::string> getAllUsers() {
        auto results = db->executeQuery("SELECT * FROM users");
        std::vector<std::string> users;
        for (const auto& row : results) {
            users.push_back(row[1] + " (" + row[2] + " years)");
        }
        return users;
    }
};

#endif
```

Part II

STL Mastery — Containers, Algorithms, Iterators

Chapter 5

STL Containers Deep Dive

STL Container Categories

1. **Sequence:** vector, deque, list, forward_list, array
2. **Associative:** set, multiset, map, multimap (sorted, tree-based)
3. **Unordered:** unordered_set, unordered_map (hash-based)
4. **Container Adapters:** stack, queue, priority_queue

5.1 Complete Container Comparison

Listing 5.1: stl_containers_master.hpp — All Containers

```
#ifndef STL_CONTAINERS_MASTER_HPP
#define STL_CONTAINERS_MASTER_HPP

#include <iostream>
#include <vector>
#include <deque>
#include <list>
#include <array>
#include <set>
#include <map>
#include <unordered_set>
#include <unordered_map>
#include <stack>
#include <queue>
#include <string>
#include <chrono>
#include <random>

//
```

```
// PERFORMANCE TESTER: Compare containers
//


template<typename Container>
class ContainerBenchmark {
public:
    static void testInsert(int n) {
        Container c;
        auto start = std::chrono::high_resolution_clock::now();

        for (int i = 0; i < n; i++) {
            insertImpl(c, i);
        }

        auto end = std::chrono::high_resolution_clock::now();
        auto ms = std::chrono::duration_cast<std::chrono::
           milliseconds>(end - start);
        std::cout << "Insert " << n << " items: " << ms.count() <<
           "ms\n";
    }

private:
    // Specializations for different containers
    static void insertImpl(std::vector<int>& c, int val) { c.
       push_back(val); }
    static void insertImpl(std::deque<int>& c, int val) { c.
       push_back(val); }
    static void insertImpl(std::list<int>& c, int val) { c.
       push_back(val); }
    static void insertImpl(std::set<int>& c, int val) { c.insert(
       val); }
    static void insertImpl(std::unordered_set<int>& c, int val) { c
       .insert(val); }
};

//


// CUSTOM HASH: For complex keys in unordered containers
//


struct Person {
    std::string name;
    int age;

    bool operator==(const Person& other) const {
        return name == other.name && age == other.age;
```

```
    }
};

// Custom hash function
struct PersonHash {
    std::size_t operator()(const Person& p) const {
        return std::hash<std::string>()(p.name) ^
               (std::hash<int>()(p.age) << 1);
    }
};

// Custom comparator for ordering
struct PersonCompare {
    bool operator()(const Person& a, const Person& b) const {
        if (a.age != b.age) return a.age < b.age;
        return a.name < b.name;
    }
};

class AdvancedContainerDemo {
public:
    static void vectorDemo() {
        std::cout << "\n=== VECTOR DEMO ===\n";

        // Capacity vs Size
        std::vector<int> v;
        std::cout << "Empty: size=" << v.size() << ", capacity=" << v.capacity() << "\n";

        v.reserve(100);  // Pre-allocate memory
        std::cout << "After reserve(100): capacity=" << v.capacity() << "\n";

        // Emplace vs Push_back
        struct Item { int x, y; Item(int a, int b) : x(a), y(b) {} };
        std::vector<Item> items;
        items.emplace_back(1, 2);  // Constructs in-place (faster)
        // vs items.push_back(Item(1, 2));  // Creates temp, then moves

        // Shrink to fit
        v.push_back(1); v.push_back(2);
        v.shrink_to_fit();  // Release excess capacity

        // Erase-remove idiom
        v = {1, 2, 3, 4, 5, 3, 3, 6};
        v.erase(std::remove(v.begin(), v.end(), 3), v.end());
        // Result: {1, 2, 4, 5, 6}
```

```
    }

    static void mapDemo() {
        std::cout << "\n=== MAP DEMO ===\n";

        // Custom key with comparator
        std::map<Person, std::string, PersonCompare> employeeDept;
        employeeDept[{"Rahul", 25}] = "Engineering";
        employeeDept[{"Priya", 30}] = "Marketing";
        employeeDept[{"Amit", 25}] = "Sales";  // Same age, sorted by name

        // Lower/upper bound
        auto it = employeeDept.lower_bound({"", 25});  // First with age >= 25

        // C++17: try_emplace, insert_or_assign
        employeeDept.try_emplace({"New", 22}, "Intern");  // Only if not exists
        employeeDept.insert_or_assign({"Rahul", 25}, "Senior Engineering");
    }

    static void unorderedDemo() {
        std::cout << "\n=== UNORDERED MAP DEMO ===\n";

        std::unordered_map<Person, double, PersonHash> salaries;
        salaries[{"Rahul", 25}] = 50000.0;
        salaries[{"Priya", 30}] = 75000.0;

        // Bucket interface
        std::cout << "Bucket count: " << salaries.bucket_count() << "\n";
        std::cout << "Load factor: " << salaries.load_factor() << "\n";

        salaries.reserve(1000);  // Pre-allocate buckets
        salaries.max_load_factor(0.7);  // Rehash threshold
    }

    static void priorityQueueDemo() {
        std::cout << "\n=== PRIORITY QUEUE DEMO ===\n";

        // Custom comparator: Min heap (default is max)
        auto cmp = [](int a, int b) { return a > b; };
        std::priority_queue<int, std::vector<int>, decltype(cmp)> minHeap(cmp);

        // Or for complex objects
```

```
        struct Task {
            int priority;
            std::string name;
            bool operator>(const Task& other) const {
                return priority > other.priority;  // Lower number
                    = higher priority
            }
        };

        std::priority_queue<Task, std::vector<Task>, std::greater<
            Task>> tasks;
        tasks.push({3, "Email"});
        tasks.push({1, "Critical Bug"});
        tasks.push({2, "Meeting"});

        while (!tasks.empty()) {
            auto t = tasks.top(); tasks.pop();
            std::cout << t.priority << ": " << t.name << "\n";
        }
    }
};

#endif
```

Chapter 6

STL Algorithms Mastery

Listing 6.1: stl_algorithms_advanced.hpp — Complete Algorithm Toolkit

```
#ifndef STL_ALGORITHMS_ADVANCED_HPP
#define STL_ALGORITHMS_ADVANCED_HPP

#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <functional>
#include <iterator>
#include <random>
#include <chrono>

class AlgorithmMastery {
public:
    //


    // SORTING & SEARCHING
    //


    static void sortingDemo() {
        std::vector<int> v = {5, 2, 8, 1, 9, 3, 7, 4, 6};

        // Partial sort: Top 3 elements sorted at front
        std::partial_sort(v.begin(), v.begin() + 3, v.end(), std::
            greater<int>());
        // v: {9, 8, 7, ...} first 3 are largest, rest unspecified

        // Nth element: Quickselect algorithm
        v = {5, 2, 8, 1, 9, 3, 7, 4, 6};
        std::nth_element(v.begin(), v.begin() + 4, v.end());  //
            Median at position 4

```

```
        // Binary search on sorted range
        std::sort(v.begin(), v.end());
        bool exists = std::binary_search(v.begin(), v.end(), 5);

        // Lower/upper bound
        auto lower = std::lower_bound(v.begin(), v.end(), 5);  // First >= 5
        auto upper = std::upper_bound(v.begin(), v.end(), 5);  // First > 5

        // Equal range
        auto [eq_lower, eq_upper] = std::equal_range(v.begin(), v.end(), 5);
    }

    //


    // TRANSFORMATIONS & FOLDING
    //


    static void transformationDemo() {
        std::vector<int> a = {1, 2, 3, 4, 5};
        std::vector<int> b = {10, 20, 30, 40, 50};
        std::vector<int> result(a.size());

        // Transform: Unary operation
        std::transform(a.begin(), a.end(), result.begin(),
                       [](int x) { return x * x; });

        // Transform: Binary operation (zip with)
        std::transform(a.begin(), a.end(), b.begin(), result.begin(),
                       [](int x, int y) { return x + y; });

        // Reduce (C++17) / Accumulate
        int sum = std::accumulate(a.begin(), a.end(), 0);
        int product = std::accumulate(a.begin(), a.end(), 1, std::multiplies<int>());

        // Inner product (dot product)
        int dot = std::inner_product(a.begin(), a.end(), b.begin(), 0);

        // Adjacent difference
        std::vector<int> diff(a.size());
        std::adjacent_difference(a.begin(), a.end(), diff.begin());
    }
```

```

    //
    // SET OPERATIONS (on sorted ranges)
    //
    static void setOperationsDemo() {
        std::vector<int> set1 = {1, 2, 3, 4, 5};
        std::vector<int> set2 = {4, 5, 6, 7, 8};
        std::vector<int> result;

        // Union
        std::set_union(set1.begin(), set1.end(),
                       set2.begin(), set2.end(),
                       std::back_inserter(result));
        // {1, 2, 3, 4, 5, 6, 7, 8}

        // Intersection
        result.clear();
        std::set_intersection(set1.begin(), set1.end(),
                              set2.begin(), set2.end(),
                              std::back_inserter(result));
        // {4, 5}

        // Difference
        result.clear();
        std::set_difference(set1.begin(), set1.end(),
                            set2.begin(), set2.end(),
                            std::back_inserter(result));
        // {1, 2, 3}

        // Symmetric difference
        result.clear();
        std::set_symmetric_difference(set1.begin(), set1.end(),
                                      set2.begin(), set2.end(),
                                      std::back_inserter(result));
        // {1, 2, 3, 6, 7, 8}
    }

    //
    // PERMUTATIONS & COMBINATORICS
    //
    static void permutationDemo() {
```

```
        std::vector<int> v = {1, 2, 3};

        // Generate all permutations
        do {
            for (int x : v) std::cout << x << " ";
            std::cout << "\n";
        } while (std::next_permutation(v.begin(), v.end()));

        // Random shuffle
        std::random_device rd;
        std::mt19937 g(rd());
        std::shuffle(v.begin(), v.end(), g);

        // Random sample
        std::vector<int> sample(2);
        std::sample(v.begin(), v.end(), sample.begin(), 2, g);
    }

    //


    // MEMORY & INITIALIZATION
    //


    static void memoryDemo() {
        std::vector<int> v(10);

        // Fill
        std::fill(v.begin(), v.end(), 42);

        // Iota: Fill with increasing sequence
        std::iota(v.begin(), v.end(), 1);  // {1, 2, 3, ..., 10}

        // Generate
        int counter = 0;
        std::generate(v.begin(), v.end(), [&counter]() { return
            counter++ * 2; });

        // Replace
        std::replace(v.begin(), v.end(), 4, 100);  // Replace all 4
            s with 100

        // Replace if
        std::replace_if(v.begin(), v.end(),
                        [](int x) { return x > 10; }, 0);
    }
};

```

```
#endif
```

Part III

4 New Complete Projects

Chapter 7

Project 1: Task Management System (Kanban Board)

Features

- **OOP:** Inheritance (Task → BugTask, FeatureTask, EpicTask), Polymorphism
- **STL:** vector, map, priority_queue, set
- **Modern C++:** Smart pointers, Lambdas, Optional, Variadic templates
- **Design Patterns:** Observer, Factory, Strategy

Listing 7.1: task_system.hpp — Complete Project 1

```cpp
#ifndef TASK_SYSTEM_HPP
#define TASK_SYSTEM_HPP

#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <queue>
#include <memory>
#include <string>
#include <chrono>
#include <optional>
#include <functional>
#include <algorithm>
#include <iomanip>

//
// FORWARD DECLARATIONS
//
```

```
class Task;
class User;
class Project;
class NotificationService;

//
// ENUMS & UTILITIES
//

enum class TaskStatus { TODO, IN_PROGRESS, IN_REVIEW, DONE,
    CANCELLED };
enum class TaskPriority { LOW, MEDIUM, HIGH, CRITICAL };
enum class TaskType { BUG, FEATURE, EPIC, STORY, SUBTASK };

inline std::string statusToString(TaskStatus s) {
    switch(s) {
        case TaskStatus::TODO: return "TODO";
        case TaskStatus::IN_PROGRESS: return "IN_PROGRESS";
        case TaskStatus::IN_REVIEW: return "IN_REVIEW";
        case TaskStatus::DONE: return "DONE";
        default: return "CANCELLED";
    }
}

inline std::string priorityToString(TaskPriority p) {
    switch(p) {
        case TaskPriority::LOW: return "LOW";
        case TaskPriority::MEDIUM: return "MEDIUM";
        case TaskPriority::HIGH: return "HIGH";
        default: return "CRITICAL";
    }
}

//
// OBSERVER PATTERN: Notification System
//

class TaskObserver {
public:
    virtual ~TaskObserver() = default;
    virtual void onTaskCreated(const Task& task) = 0;
    virtual void onTaskUpdated(const Task& task, const std::string&
```

```
        field) = 0;
    virtual void onTaskCompleted(const Task& task) = 0;
    virtual void onTaskAssigned(const Task& task, const User& user) = 0;
};

//


// USER CLASS
//


class User {
private:
    int id;
    std::string name;
    std::string email;
    std::string role;  // Developer, Manager, QA, etc.
    std::set<int> assignedTaskIds;

public:
    User(int i, std::string n, std::string e, std::string r)
        : id(i), name(n), email(e), role(r) {}

    int getId() const { return id; }
    std::string getName() const { return name; }
    std::string getEmail() const { return email; }
    std::string getRole() const { return role; }

    void assignTask(int taskId) { assignedTaskIds.insert(taskId); }
    void unassignTask(int taskId) { assignedTaskIds.erase(taskId); }
    bool hasTask(int taskId) const {
        return assignedTaskIds.find(taskId) != assignedTaskIds.end();
    }

    friend std::ostream& operator<<(std::ostream& os, const User& u) {
        os << "[" << u.id << "] " << u.name << " (" << u.role << ")";
        return os;
    }
};

//
```

```
// ABSTRACT BASE: Task (Polymorphism)
//


class Task {
protected:
    int id;
    std::string title;
    std::string description;
    TaskStatus status;
    TaskPriority priority;
    TaskType type;

    int creatorId;
    std::optional<int> assigneeId;

    std::chrono::system_clock::time_point createdAt;
    std::optional<std::chrono::system_clock::time_point> startedAt;
    std::optional<std::chrono::system_clock::time_point>
       completedAt;

    std::vector<std::string> comments;
    std::set<int> blockedBy;  // Task IDs that block this task
    std::set<int> blocks;     // Task IDs this task blocks

    static int nextId;
    std::vector<TaskObserver*> observers;

public:
    Task(std::string t, std::string d, TaskPriority p,
         TaskType ty, int creator)
        : id(nextId++), title(t), description(d),
          status(TaskStatus::TODO), priority(p), type(ty),
          creatorId(creator), assigneeId(std::nullopt),
          createdAt(std::chrono::system_clock::now()) {}

    virtual ~Task() = default;

    // Getters
    int getId() const { return id; }
    std::string getTitle() const { return title; }
    TaskStatus getStatus() const { return status; }
    TaskPriority getPriority() const { return priority; }
    TaskType getType() const { return type; }
    std::optional<int> getAssigneeId() const { return assigneeId; }

    // Virtual methods for polymorphism
    virtual double getEstimatedHours() const = 0;
    virtual std::string getTaskDetails() const = 0;
```

```
    virtual bool canStart() const {
        return status == TaskStatus::TODO && blockedBy.empty();
    }

    // Observer pattern
    void addObserver(TaskObserver* obs) { observers.push_back(obs);
        }
    void removeObserver(TaskObserver* obs);

    void notifyCreated() {
        for (auto* obs : observers) obs->onTaskCreated(*this);
    }
    void notifyUpdated(const std::string& field) {
        for (auto* obs : observers) obs->onTaskUpdated(*this, field
            );
    }

    // Status transitions
    bool start() {
        if (!canStart()) return false;
        status = TaskStatus::IN_PROGRESS;
        startedAt = std::chrono::system_clock::now();
        notifyUpdated("status");
        return true;
    }

    bool complete() {
        if (status != TaskStatus::IN_PROGRESS &&
            status != TaskStatus::IN_REVIEW) return false;
        status = TaskStatus::DONE;
        completedAt = std::chrono::system_clock::now();
        for (auto* obs : observers) obs->onTaskCompleted(*this);
        return true;
    }

    void assignTo(int userId) {
        assigneeId = userId;
        notifyUpdated("assignee");
    }

    void addComment(const std::string& comment) {
        auto now = std::chrono::system_clock::now();
        auto time = std::chrono::system_clock::to_time_t(now);
        std::stringstream ss;
        ss << std::put_time(std::localtime(&time), "%Y-%m-%d %H:%M"
            );
        comments.push_back("[" + ss.str() + "] " + comment);
    }

```

```
    void addDependency(int blocksThisTask) {
        blockedBy.insert(blocksThisTask);
    }

    // For priority queue comparison
    bool operator<(const Task& other) const {
        // Higher priority comes first
        if (priority != other.priority)
            return priority < other.priority;
        return createdAt > other.createdAt;  // Older first
    }

    friend std::ostream& operator<<(std::ostream& os, const Task& t
        ) {
        os << "[" << t.id << "] " << t.title
           << " [" << statusToString(t.status) << "]"
           << " (" << priorityToString(t.priority) << ")";
        if (t.assigneeId) os << " -> User " << *t.assigneeId;
        return os;
    }
};

int Task::nextId = 1;

//


// CONCRETE TASK TYPES (Inheritance + Polymorphism)
//



class BugTask : public Task {
private:
    std::string severity;  // Critical, Major, Minor, Trivial
    std::string environment;  // Production, Staging, Dev
    std::optional<int> relatedFeatureId;

public:
    BugTask(std::string t, std::string d, TaskPriority p, int
        creator,
            std::string sev, std::string env)
        : Task(t, d, p, TaskType::BUG, creator),
          severity(sev), environment(env) {}

    double getEstimatedHours() const override {
        if (severity == "Critical") return 4.0;
        if (severity == "Major") return 8.0;
        if (severity == "Minor") return 2.0;
```

```
        return 1.0;
    }

    std::string getTaskDetails() const override {
        return "BUG [" + severity + "] in " + environment;
    }
};

class FeatureTask : public Task {
private:
    std::string component;  // UI, Backend, Database, etc.
    std::vector<std::string> acceptanceCriteria;
    int storyPoints;  // Fibonacci: 1, 2, 3, 5, 8, 13...

public:
    FeatureTask(std::string t, std::string d, TaskPriority p, int creator,
                std::string comp, int points)
        : Task(t, d, p, TaskType::FEATURE, creator),
          component(comp), storyPoints(points) {}

    double getEstimatedHours() const override {
        return storyPoints * 4.0;  // 1 point = 4 hours
    }

    std::string getTaskDetails() const override {
        return "FEATURE [" + component + "] " +
               std::to_string(storyPoints) + " points";
    }

    void addAcceptanceCriteria(const std::string& criteria) {
        acceptanceCriteria.push_back(criteria);
    }
};

class EpicTask : public Task {
private:
    std::vector<int> storyIds;  // Child stories
    std::string businessGoal;
    double targetROI;

public:
    EpicTask(std::string t, std::string d, TaskPriority p, int creator,
             std::string goal, double roi)
        : Task(t, d, p, TaskType::EPIC, creator),
          businessGoal(goal), targetROI(roi) {}

    double getEstimatedHours() const override {
```

```
        double total = 0;
        // Would lookup stories and sum their estimates
        return total;
    }

    std::string getTaskDetails() const override {
        return "EPIC: " + businessGoal + " (ROI: " +
               std::to_string(targetROI) + "%)";
    }

    void addStory(int storyId) { storyIds.push_back(storyId); }
};

//


// FACTORY PATTERN: Task Creation
//


class TaskFactory {
public:
    template<typename... Args>
    static std::unique_ptr<Task> createBug(Args&&... args) {
        return std::make_unique<BugTask>(std::forward<Args>(args)
            ...);
    }

    template<typename... Args>
    static std::unique_ptr<Task> createFeature(Args&&... args) {
        return std::make_unique<FeatureTask>(std::forward<Args>(
            args)...);
    }

    template<typename... Args>
    static std::unique_ptr<Task> createEpic(Args&&... args) {
        return std::make_unique<EpicTask>(std::forward<Args>(args)
            ...);
    }
};

//


// PROJECT / BOARD CLASS
//


class KanbanBoard {
```

```
private:
    std::string name;
    std::map<int, std::unique_ptr<Task>> tasks;  // ID -> Task
    std::map<int, std::unique_ptr<User>> users;   // ID -> User

    // Columns (status -> ordered tasks)
    std::map<TaskStatus, std::set<int>> columns;

    // Priority queue for urgent tasks
    std::priority_queue<std::pair<TaskPriority, int>> urgentTasks;

    std::vector<TaskObserver*> globalObservers;

public:
    explicit KanbanBoard(std::string n) : name(n) {}

    // User management
    void addUser(std::unique_ptr<User> user) {
        users[user->getId()] = std::move(user);
    }

    // Task creation with factory
    template<typename... Args>
    Task& createBug(Args&&... args) {
        auto task = TaskFactory::createBug(std::forward<Args>(args)
            ...);
        int id = task->getId();
        tasks[id] = std::move(task);
        columns[TaskStatus::TODO].insert(id);
        notifyTaskCreated(*tasks[id]);
        return *tasks[id];
    }

    template<typename... Args>
    Task& createFeature(Args&&... args) {
        auto task = TaskFactory::createFeature(std::forward<Args>(
            args)...);
        int id = task->getId();
        tasks[id] = std::move(task);
        columns[TaskStatus::TODO].insert(id);
        notifyTaskCreated(*tasks[id]);
        return *tasks[id];
    }

    // Task operations
    bool moveTask(int taskId, TaskStatus newStatus) {
        auto it = tasks.find(taskId);
        if (it == tasks.end()) return false;

```

```
        TaskStatus oldStatus = it->second->getStatus();
        columns[oldStatus].erase(taskId);
        columns[newStatus].insert(taskId);

        // Update task internal status
        if (newStatus == TaskStatus::IN_PROGRESS) {
            it->second->start();
        } else if (newStatus == TaskStatus::DONE) {
            it->second->complete();
        }

        return true;
    }

    bool assignTask(int taskId, int userId) {
        auto taskIt = tasks.find(taskId);
        auto userIt = users.find(userId);
        if (taskIt == tasks.end() || userIt == users.end()) return
           false;

        // Unassign from previous
        if (auto prevId = taskIt->second->getAssigneeId()) {
            users[*prevId]->unassignTask(taskId);
        }

        taskIt->second->assignTo(userId);
        userIt->second->assignTask(taskId);

        for (auto* obs : globalObservers) {
            obs->onTaskAssigned(*taskIt->second, *userIt->second);
        }
        return true;
    }

    // Display
    void displayBoard() const {
        std::cout << "\n=== " << name << " ===\n";
        for (const auto& [status, taskIds] : columns) {
            std::cout << "\n[" << statusToString(status) << "] "
                      << taskIds.size() << " tasks\n";
            for (int id : taskIds) {
                if (auto it = tasks.find(id); it != tasks.end()) {
                    std::cout << "  " << *it->second << "\n";
                }
            }
        }
    }

    void displayUserWorkload() const {
```

```
        std::cout << "\n=== User Workload ===\n";
        for (const auto& [id, user] : users) {
            int count = 0;
            for (const auto& [tid, task] : tasks) {
                if (auto assignee = task->getAssigneeId();
                    assignee && *assignee == id) {
                    count++;
                }
            }
            std::cout << *user << ": " << count << " tasks\n";
        }
    }

    // Observer
    void addGlobalObserver(TaskObserver* obs) {
        globalObservers.push_back(obs);
    }

private:
    void notifyTaskCreated(const Task& task) {
        for (auto* obs : globalObservers) obs->onTaskCreated(task);
    }
};

//


// CONCRETE OBSERVER: Email Notifications
//


class EmailNotificationService : public TaskObserver {
public:
    void onTaskCreated(const Task& task) override {
        std::cout << "[EMAIL] New task created: " << task.getTitle
            () << "\n";
    }

    void onTaskUpdated(const Task& task, const std::string& field)
        override {
        std::cout << "[EMAIL] Task " << task.getId() << " updated:
            "
                  << field << "\n";
    }

    void onTaskCompleted(const Task& task) override {
        std::cout << "[EMAIL]      Task completed: " << task.
            getTitle() << "\n";
    }
```

```cpp

    void onTaskAssigned(const Task& task, const User& user) override {
        std::cout << "[EMAIL] Task assigned to " << user.getName() << "\n";
    }
};

#endif
```

Listing 7.2: task_main.cpp — Project 1 Demo

```cpp
#include "task_system.hpp"

int main() {
    // Create board
    KanbanBoard board("Sprint 42");

    // Add notification service
    EmailNotificationService emailService;
    board.addGlobalObserver(&emailService);

    // Add users
    board.addUser(std::make_unique<User>(1, "Rahul", "rahul@company.com", "Senior Dev"));
    board.addUser(std::make_unique<User>(2, "Priya", "priya@company.com", "Tech Lead"));
    board.addUser(std::make_unique<User>(3, "Amit", "amit@company.com", "QA Engineer"));

    // Create tasks using factory
    auto& bug1 = board.createBug("Login crash on mobile",
                                 "App crashes when clicking login",
                                 TaskPriority::CRITICAL, 1, "Critical", "Production");

    auto& feature1 = board.createFeature("Dark mode support",
                                         "Add dark theme to all screens",
                                         TaskPriority::HIGH, 1,
                                         "UI", 5);

    auto& feature2 = board.createFeature("API rate limiting",
                                         "Implement token bucket algorithm",
                                         TaskPriority::MEDIUM, 2,
                                         "Backend", 8);

    // Assign tasks
```

```
    board.assignTask(bug1.getId(), 1);
    board.assignTask(feature1.getId(), 1);
    board.assignTask(feature2.getId(), 2);

    // Move tasks through workflow
    board.moveTask(bug1.getId(), TaskStatus::IN_PROGRESS);
    board.moveTask(feature2.getId(), TaskStatus::IN_PROGRESS);

    // Complete a task
    board.moveTask(bug1.getId(), TaskStatus::IN_REVIEW);
    board.moveTask(bug1.getId(), TaskStatus::DONE);

    // Display
    board.displayBoard();
    board.displayUserWorkload();

    // Demonstrate polymorphism
    std::cout << "\n=== Task Details (Polymorphism) ===\n";
    std::cout << bug1.getTaskDetails() << " - "
              << bug1.getEstimatedHours() << "h\n";
    std::cout << feature1.getTaskDetails() << " - "
              << feature1.getEstimatedHours() << "h\n";

    return 0;
}
```

Chapter 8

Project 2: E-Commerce Platform with Inventory

Features

- **OOP:** Multiple inheritance (Product variants), Abstract factory
- **STL:** map, multimap, priority_queue, algorithm (sorting, searching)
- **Modern C++:** std::variant, std::optional, std::expected (C++23), Concepts
- **Design Patterns:** Strategy (pricing), Decorator (discounts), Singleton (cart)

Listing 8.1: ecommerce_platform.hpp — Complete Project 2

```
#ifndef ECOMMERCE_PLATFORM_HPP
#define ECOMMERCE_PLATFORM_HPP

#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <memory>
#include <variant>
#include <optional>
#include <functional>
#include <algorithm>
#include <numeric>
#include <chrono>
#include <iomanip>
#include <sstream>
#include <random>

//
```

```
// FORWARD DECLARATIONS
//

class Product;
class Cart;
class Order;
class PricingStrategy;
class DiscountDecorator;

//

// MONEY TYPE (Precision handling)
//

class Money {
private:
    long long cents;  // Store as integer to avoid floating point errors

public:
    explicit Money(double amount) : cents(static_cast<long long>(amount * 100)) {}
    explicit Money(long long c) : cents(c) {}

    double getAmount() const { return cents / 100.0; }
    long long getCents() const { return cents; }

    Money operator+(const Money& other) const { return Money(cents + other.cents); }
    Money operator-(const Money& other) const { return Money(cents - other.cents); }
    Money operator*(int qty) const { return Money(cents * qty); }
    Money operator*(double factor) const { return Money(static_cast<long long>(cents * factor)); }

    bool operator<(const Money& other) const { return cents < other.cents; }
    bool operator>(const Money& other) const { return cents > other.cents; }
    bool operator==(const Money& other) const { return cents == other.cents; }

    Money applyDiscount(double percent) const {
        return Money(static_cast<long long>(cents * (1 - percent)));
    }
```

```

    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        os << "$" << std::fixed << std::setprecision(2) << m.getAmount();
        return os;
    }
};

//


// STRATEGY PATTERN: Pricing Strategies
//


class PricingStrategy {
public:
    virtual ~PricingStrategy() = default;
    virtual Money calculatePrice(const Money& basePrice, int quantity) const = 0;
    virtual std::string getDescription() const = 0;
};

class RegularPricing : public PricingStrategy {
public:
    Money calculatePrice(const Money& basePrice, int quantity) const override {
        return basePrice * quantity;
    }
    std::string getDescription() const override { return "Regular"; }
};

class BulkPricing : public PricingStrategy {
private:
    int threshold;
    double discountPercent;

public:
    BulkPricing(int t, double d) : threshold(t), discountPercent(d) {}

    Money calculatePrice(const Money& basePrice, int quantity) const override {
        Money total = basePrice * quantity;
        if (quantity >= threshold) {
            return total.applyDiscount(discountPercent);
        }
```

```
        return total;
    }
    std::string getDescription() const override {
        return "Bulk (>" + std::to_string(threshold) + " = " +
                std::to_string(static_cast<int>(discountPercent *
                    100)) + "% off)";
    }
};

class SeasonalPricing : public PricingStrategy {
private:
    double multiplier;
    std::string season;

public:
    SeasonalPricing(double m, std::string s) : multiplier(m),
        season(s) {}

    Money calculatePrice(const Money& basePrice, int quantity)
        const override {
        return Money(basePrice.getAmount() * multiplier * quantity)
            ;
    }
    std::string getDescription() const override { return season + "
        Sale"; }
};

//


// DECORATOR PATTERN: Discounts
//


class DiscountDecorator : public PricingStrategy {
protected:
    std::unique_ptr<PricingStrategy> wrapped;

public:
    explicit DiscountDecorator(std::unique_ptr<PricingStrategy> w)
        : wrapped(std::move(w)) {}

    virtual Money applyDiscountLogic(const Money& price) const = 0;

    Money calculatePrice(const Money& basePrice, int quantity)
        const override {
        Money base = wrapped->calculatePrice(basePrice, quantity);
        return applyDiscountLogic(base);
    }
```

```

    std::string getDescription() const override {
        return wrapped->getDescription() + " + Discount";
    }
};

class CouponDiscount : public DiscountDecorator {
private:
    std::string code;
    double percentOff;

public:
    CouponDiscount(std::unique_ptr<PricingStrategy> w, std::string
       c, double p)
        : DiscountDecorator(std::move(w)), code(c), percentOff(p)
           {}

    Money applyDiscountLogic(const Money& price) const override {
        return price.applyDiscount(percentOff);
    }

    std::string getDescription() const override {
        return wrapped->getDescription() + " + Coupon " + code;
    }
};

class MemberDiscount : public DiscountDecorator {
private:
    int loyaltyYears;

public:
    MemberDiscount(std::unique_ptr<PricingStrategy> w, int years)
        : DiscountDecorator(std::move(w)), loyaltyYears(years) {}

    Money applyDiscountLogic(const Money& price) const override {
        double discount = std::min(0.05 + (loyaltyYears * 0.01),
           0.20);
        return price.applyDiscount(discount);
    }
};

//


// PRODUCT HIERARCHY (Inheritance + Polymorphism)
//


class Product {
```

```
protected:
    int id;
    std::string name;
    std::string description;
    std::string category;
    Money basePrice;
    int stockQuantity;
    std::string sku;  // Stock Keeping Unit

    std::unique_ptr<PricingStrategy> pricingStrategy;

public:
    Product(int i, std::string n, std::string d, std::string c,
            Money price, int stock, std::string s)
        : id(i), name(n), description(d), category(c),
          basePrice(price), stockQuantity(stock), sku(s),
          pricingStrategy(std::make_unique<RegularPricing>()) {}

    virtual ~Product() = default;

    // Getters
    int getId() const { return id; }
    std::string getName() const { return name; }
    std::string getCategory() const { return category; }
    Money getBasePrice() const { return basePrice; }
    int getStock() const { return stockQuantity; }
    std::string getSku() const { return sku; }

    // Pricing
    void setPricingStrategy(std::unique_ptr<PricingStrategy>
       strategy) {
        pricingStrategy = std::move(strategy);
    }

    Money getPrice(int quantity = 1) const {
        return pricingStrategy->calculatePrice(basePrice, quantity)
           ;
    }

    // Stock management
    bool reserveStock(int qty) {
        if (qty > stockQuantity) return false;
        stockQuantity -= qty;
        return true;
    }

    void releaseStock(int qty) {
        stockQuantity += qty;
    }
```

```

    // Virtual methods for variants
    virtual std::string getProductType() const = 0;
    virtual std::vector<std::string> getSpecifications() const = 0;
    virtual bool isInStock() const { return stockQuantity > 0; }

    // For inventory sorting
    bool operator<(const Product& other) const { return name < other.name; }

    friend std::ostream& operator<<(std::ostream& os, const Product& p) {
        os << "[" << p.sku << "] " << p.name << " " << p.basePrice
           << " (" << p.stockQuantity << " in stock)";
        return os;
    }
};

//


// CONCRETE PRODUCT TYPES
//



class PhysicalProduct : public Product {
private:
    double weightKg;
    std::string dimensions;
    std::string warehouseLocation;

public:
    PhysicalProduct(int i, std::string n, std::string d, std::string c,
                    Money price, int stock, std::string s,
                    double w, std::string dim, std::string loc)
        : Product(i, n, d, c, price, stock, s),
          weightKg(w), dimensions(dim), warehouseLocation(loc) {}

    std::string getProductType() const override { return "Physical"; }

    std::vector<std::string> getSpecifications() const override {
        return {"Weight: " + std::to_string(weightKg) + "kg",
                "Dimensions: " + dimensions,
                "Location: " + warehouseLocation};
    }

```

```
    double getShippingCost(const std::string& destination) const {
        // Simple weight-based calculation
        return weightKg * 2.5;  // $2.5 per kg
    }
};

class DigitalProduct : public Product {
private:
    std::string downloadUrl;
    std::string fileFormat;
    double fileSizeMB;
    int downloadLimit;

public:
    DigitalProduct(int i, std::string n, std::string d, std::string c,
                   Money price, std::string s,
                   std::string url, std::string fmt, double size)
        : Product(i, n, d, c, price, 999999, s),  // Unlimited stock
          downloadUrl(url), fileFormat(fmt), fileSizeMB(size),
          downloadLimit(3) {}

    std::string getProductType() const override { return "Digital"; }

    std::vector<std::string> getSpecifications() const override {
        return {"Format: " + fileFormat,
                "Size: " + std::to_string(fileSizeMB) + "MB",
                "Downloads: " + std::to_string(downloadLimit)};
    }

    bool isInStock() const override { return true; }  // Always available

    std::string generateDownloadLink(int orderId) const {
        return downloadUrl + "?order=" + std::to_string(orderId) +
               "&token=" + generateToken();
    }

private:
    std::string generateToken() const {
        // Simple random token
        std::random_device rd;
        std::mt19937 gen(rd());
        std::uniform_int_distribution<> dis(100000, 999999);
        return std::to_string(dis(gen));
    }
};
```

```

class SubscriptionProduct : public Product {
private:
    std::string billingCycle;  // monthly, yearly
    int trialDays;
    std::vector<std::string> includedFeatures;

public:
    SubscriptionProduct(int i, std::string n, std::string d, std::string c,
                        Money price, std::string s,
                        std::string cycle, int trial)
        : Product(i, n, d, c, price, 999999, s),
          billingCycle(cycle), trialDays(trial) {}

    std::string getProductType() const override { return "Subscription"; }

    std::vector<std::string> getSpecifications() const override {
        return {"Billing: " + billingCycle,
                "Trial: " + std::to_string(trialDays) + " days"};
    }

    Money getTrialPrice() const {
        return Money(0.0);
    }

    void addFeature(const std::string& feature) {
        includedFeatures.push_back(feature);
    }
};

//


// INVENTORY MANAGEMENT (STL heavy)
//


class Inventory {
private:
    // Multiple indices for fast lookup
    std::map<int, std::unique_ptr<Product>> byId;
    std::map<std::string, std::set<int>> byCategory;
    std::map<std::string, int> bySku;  // SKU -> ID
    std::multimap<Money, int> byPrice;  // Can have same price

    // Search index: word -> product IDs
    std::map<std::string, std::set<int>> searchIndex;
```

```

public:
    void addProduct(std::unique_ptr<Product> product) {
        int id = product->getId();
        std::string sku = product->getSku();
        std::string category = product->getCategory();
        Money price = product->getBasePrice();

        // Update all indices
        byId[id] = std::move(product);
        byCategory[category].insert(id);
        bySku[sku] = id;
        byPrice.insert({price, id});

        // Update search index
        updateSearchIndex(id, byId[id]->getName());
        updateSearchIndex(id, byId[id]->getCategory());
    }

    Product* findById(int id) {
        auto it = byId.find(id);
        return (it != byId.end()) ? it->second.get() : nullptr;
    }

    Product* findBySku(const std::string& sku) {
        auto it = bySku.find(sku);
        if (it == bySku.end()) return nullptr;
        return findById(it->second);
    }

    std::vector<Product*> findByCategory(const std::string&
       category) {
        std::vector<Product*> results;
        auto it = byCategory.find(category);
        if (it != byCategory.end()) {
            for (int id : it->second) {
                results.push_back(findById(id));
            }
        }
        return results;
    }

    std::vector<Product*> search(const std::string& query) {
        std::set<int> resultIds;
        std::istringstream iss(query);
        std::string word;

        while (iss >> word) {
            // Convert to lowercase
```

```
            std::transform(word.begin(), word.end(), word.begin(), ::tolower);

            auto it = searchIndex.find(word);
            if (it != searchIndex.end()) {
                if (resultIds.empty()) {
                    resultIds = it->second;
                } else {
                    // Intersection (AND search)
                    std::set<int> intersection;
                    std::set_intersection(resultIds.begin(), resultIds.end(),
                                          it->second.begin(), it->second.end(),
                                          std::inserter(intersection, intersection.begin()));
                    resultIds = intersection;
                }
            }
        }

        std::vector<Product*> results;
        for (int id : resultIds) {
            results.push_back(findById(id));
        }
        return results;
    }

    std::vector<Product*> getProductsInPriceRange(Money minPrice, Money maxPrice) {
        std::vector<Product*> results;
        auto range = byPrice.lower_bound(minPrice);
        while (range != byPrice.end() && range->first < maxPrice) {
            results.push_back(findById(range->second));
            ++range;
        }
        return results;
    }

    void displayInventory() const {
        std::cout << "\n=== INVENTORY ===\n";
        for (const auto& [id, product] : byId) {
            std::cout << "  " << *product << " ["
                      << product->getProductType() << "]\n";
        }
    }

private:
    void updateSearchIndex(int id, const std::string& text) {
```

```
        std::istringstream iss(text);
        std::string word;
        while (iss >> word) {
            std::transform(word.begin(), word.end(), word.begin(),
               ::tolower);
            searchIndex[word].insert(id);
        }
    }
};

//


// CART (Singleton pattern for user session)
//


class CartItem {
public:
    Product* product;
    int quantity;
    std::unique_ptr<PricingStrategy> pricingOverride;

    CartItem(Product* p, int q) : product(p), quantity(q) {}

    Money getTotal() const {
        if (pricingOverride) {
            return pricingOverride->calculatePrice(product->
               getBasePrice(), quantity);
        }
        return product->getPrice(quantity);
    }
};

class ShoppingCart {
private:
    static std::map<int, std::unique_ptr<ShoppingCart>> userCarts;
        // UserID -> Cart

    std::vector<CartItem> items;
    std::optional<int> userId;
    std::chrono::system_clock::time_point lastActivity;

    ShoppingCart(int uid) : userId(uid), lastActivity(std::chrono::
       system_clock::now()) {}

public:
    // Singleton per user
    static ShoppingCart& getInstance(int userId) {
```

```
        if (userCarts.find(userId) == userCarts.end()) {
            userCarts[userId] = std::unique_ptr<ShoppingCart>(new ShoppingCart(userId));
        }
        return *userCarts[userId];
    }

    void addItem(Product* product, int quantity) {
        // Check if already in cart
        auto it = std::find_if(items.begin(), items.end(),
                               [product](const CartItem& item) {
                                   return item.product == product;
                               });

        if (it != items.end()) {
            it->quantity += quantity;
        } else {
            items.emplace_back(product, quantity);
        }
        updateActivity();
    }

    void removeItem(Product* product) {
        items.erase(std::remove_if(items.begin(), items.end(),
                                   [product](const CartItem& item) {
                                       return item.product == product;
                                   }), items.end());
    }

    void applyCoupon(const std::string& code, double discount) {
        // Apply to all items
        for (auto& item : items) {
            auto currentStrategy = std::make_unique<RegularPricing>();
            item.pricingOverride = std::make_unique<CouponDiscount>(
                std::move(currentStrategy), code, discount);
        }
    }

    Money getSubtotal() const {
        return std::accumulate(items.begin(), items.end(), Money(0.0),
                               [](const Money& sum, const CartItem& item) {
                                   return sum + item.getTotal();
                               });
```

```
    }

    Money getTax(double taxRate = 0.08) const {
        return Money(getSubtotal().getAmount() * taxRate);
    }

    Money getTotal(double taxRate = 0.08) const {
        return getSubtotal() + getTax(taxRate);
    }

    void display() const {
        std::cout << "\n=== SHOPPING CART ===\n";
        for (const auto& item : items) {
            std::cout << "  " << item.product->getName()
                      << " x" << item.quantity
                      << " = " << item.getTotal() << "\n";
        }
        std::cout << "  Subtotal: " << getSubtotal() << "\n";
        std::cout << "  Tax (8%): " << getTax() << "\n";
        std::cout << "  TOTAL: " << getTotal() << "\n";
    }

    bool isExpired() const {
        auto now = std::chrono::system_clock::now();
        auto hours = std::chrono::duration_cast<std::chrono::hours
           >(now - lastActivity);
        return hours.count() > 24;  // 24 hour expiry
    }

private:
    void updateActivity() {
        lastActivity = std::chrono::system_clock::now();
    }
};

// Static member definition
std::map<int, std::unique_ptr<ShoppingCart>> ShoppingCart::
   userCarts;

//


// ORDER & CHECKOUT
//


enum class OrderStatus { PENDING, PAID, PROCESSING, SHIPPED,
   DELIVERED, CANCELLED };

```

```
class Order {
private:
    int orderId;
    int userId;
    std::vector<CartItem> items;
    Money subtotal;
    Money tax;
    Money shipping;
    Money total;
    OrderStatus status;
    std::chrono::system_clock::time_point orderDate;
    std::string shippingAddress;

public:
    Order(int oid, int uid, const ShoppingCart& cart,
          std::string address, Money shipCost)
        : orderId(oid), userId(uid), items(cart.getItems()),
          subtotal(cart.getSubtotal()), tax(cart.getTax()),
          shipping(shipCost), total(cart.getTotal() + shipCost),
          status(OrderStatus::PENDING),
          orderDate(std::chrono::system_clock::now()),
          shippingAddress(address) {}

    bool processPayment(const std::string& paymentMethod) {
        // Simulate payment processing
        std::cout << "Processing " << paymentMethod
                  << " payment of " << total << "... ";
        // 90% success rate simulation
        std::random_device rd;
        std::mt19937 gen(rd());
        if (std::uniform_int_distribution<>(1, 10)(gen) <= 9) {
            status = OrderStatus::PAID;
            std::cout << "SUCCESS\n";
            return true;
        }
        std::cout << "FAILED\n";
        return false;
    }

    void fulfill() {
        if (status == OrderStatus::PAID) {
            status = OrderStatus::PROCESSING;
            // Reserve inventory, generate shipping label, etc.
        }
    }

    friend std::ostream& operator<<(std::ostream& os, const Order&
       o) {
        auto time = std::chrono::system_clock::to_time_t(o.
```

```
            orderDate);
        os << "Order #" << o.orderId << " | " << o.total
            << " | " << ctime(&time);
        return os;
    }

    // Getters for order management
    int getId() const { return orderId; }
    OrderStatus getStatus() const { return status; }
    Money getTotal() const { return total; }
};

#endif
```

Chapter 9

Project 3: Social Network Graph System

Features

- **OOP:** Complex inheritance (User → RegularUser, Influencer, Business), Friend relationships
- **STL:** Graph representation (adjacency list), BFS/DFS, set operations
- **Modern C++:** Coroutines (C++20), Ranges (C++20), std::any for flexible data
- **Design Patterns:** Observer (notifications), Memento (post history), Flyweight (emoji)

Listing 9.1: social_network.hpp — Complete Project 3

```cpp
#ifndef SOCIAL_NETWORK_HPP
#define SOCIAL_NETWORK_HPP

#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <queue>
#include <stack>
#include <memory>
#include <any>
#include <optional>
#include <variant>
#include <functional>
#include <algorithm>
#include <chrono>
#include <sstream>
#include <iomanip>
```

```

// C++20 features (if available)
#if __cplusplus >= 202002L
#include <coroutine>
#include <ranges>
#endif

//
// FORWARD DECLARATIONS
//

class User;
class Post;
class Comment;
class SocialGraph;
class NotificationManager;

//
// UTILITY: Time formatting
//

inline std::string formatTime(std::chrono::system_clock::time_point tp) {
    auto time = std::chrono::system_clock::to_time_t(tp);
    std::stringstream ss;
    ss << std::put_time(std::localtime(&time), "%Y-%m-%d %H:%M");
    return ss.str();
}

//
// CONTENT VARIANT: Different types of post content
//

using ContentData = std::variant<
    std::string,                                    // Text
    std::pair<std::string, std::string>,            // Image: (url, caption)
    std::tuple<std::string, int, int>,              // Video: (url, duration, views)
    std::map<std::string, std::string>              // Poll: option
```

```
        -> vote count
>;

//


// FLYWEIGHT PATTERN: Emoji storage (shared instances)
//


class EmojiFlyweight {
private:
    static std::map<std::string, std::weak_ptr<std::string>> pool;

public:
    static std::shared_ptr<std::string> get(const std::string&
        emoji) {
        auto it = pool.find(emoji);
        if (it != pool.end()) {
            if (auto shared = it->second.lock()) {
                return shared;
            }
        }
        auto shared = std::make_shared<std::string>(emoji);
        pool[emoji] = shared;
        return shared;
    }
};

std::map<std::string, std::weak_ptr<std::string>> EmojiFlyweight::
    pool;

//


// COMMENT CLASS
//


class Comment {
private:
    int id;
    int authorId;
    std::string text;
    std::vector<std::shared_ptr<std::string>> emojis;
    std::chrono::system_clock::time_point createdAt;
    int likes;
    std::set<int> likedBy;

```

```
public:
    Comment(int i, int author, std::string t)
        : id(i), authorId(author), text(t),
          createdAt(std::chrono::system_clock::now()), likes(0) {}

    void addEmoji(const std::string& emoji) {
        emojis.push_back(EmojiFlyweight::get(emoji));
    }

    bool like(int userId) {
        if (likedBy.insert(userId).second) {
            likes++;
            return true;
        }
        return false;  // Already liked
    }

    friend std::ostream& operator<<(std::ostream& os, const Comment
       & c) {
        os << "  [" << c.id << "] " << c.text;
        for (const auto& e : c.emojis) os << *e;
        os << " (" << c.likes << " likes)";
        return os;
    }
};

//


// POST CLASS (Memento pattern for history)
//


class PostMemento {
private:
    std::string content;
    std::chrono::system_clock::time_point savedAt;

public:
    PostMemento(std::string c)
        : content(c), savedAt(std::chrono::system_clock::now()) {}

    std::string getContent() const { return content; }
    std::string getTime() const { return formatTime(savedAt); }
};

class Post {
private:
    int id;
```

```
    int authorId;
    ContentData content;
    std::vector<Comment> comments;
    std::set<int> likedBy;
    int shares;
    std::chrono::system_clock::time_point postedAt;
    std::optional<std::chrono::system_clock::time_point> editedAt;

    // Memento history
    std::vector<PostMemento> history;

public:
    Post(int i, int author, ContentData c)
        : id(i), authorId(author), content(c), shares(0),
          postedAt(std::chrono::system_clock::now()) {
        saveMemento();  // Save initial state
    }

    void edit(ContentData newContent) {
        saveMemento();
        content = newContent;
        editedAt = std::chrono::system_clock::now();
    }

    void saveMemento() {
        std::string contentStr;
        std::visit([&contentStr](auto&& arg) {
            using T = std::decay_t<decltype(arg)>;
            if constexpr (std::is_same_v<T, std::string>) {
                contentStr = arg;
            } else {
                contentStr = "[complex content]";
            }
        }, content);
        history.emplace_back(contentStr);
    }

    void restoreMemento(size_t index) {
        if (index < history.size()) {
            content = history[index].getContent();
        }
    }

    void addComment(const Comment& comment) {
        comments.push_back(comment);
    }

    bool like(int userId) {
        return likedBy.insert(userId).second;
```

```
    }

    void share() { shares++; }

    int getEngagementScore() const {
        return likedBy.size() * 1 + comments.size() * 3 + shares *
           5;
    }

    friend std::ostream& operator<<(std::ostream& os, const Post& p
       ) {
        os << "Post #" << p.id << " by User " << p.authorId;
        if (p.editedAt) os << " [edited]";
        os << "\n  Engagement: " << p.getEngagementScore()
           << " (likes: " << p.likedBy.size()
           << ", comments: " << p.comments.size()
           << ", shares: " << p.shares << ")";
        return os;
    }

    int getId() const { return id; }
    int getAuthorId() const { return authorId; }
    int getLikes() const { return likedBy.size(); }
};

//


// USER HIERARCHY (Inheritance)
//


enum class UserType { REGULAR, INFLUENCER, BUSINESS, BOT };

class User {
protected:
    int id;
    std::string username;
    std::string displayName;
    std::string bio;
    UserType type;

    std::set<int> friends;            // Bidirectional
    std::set<int> followers;          // One-way (they follow me)
    std::set<int> following;          // One-way (I follow them)
    std::set<int> blocked;

    std::vector<Post> posts;
    std::map<int, std::any> settings;  // Flexible settings storage
```

```

    std::chrono::system_clock::time_point joinedAt;
    bool isActive;

public:
    User(int i, std::string u, std::string d, std::string b,
       UserType t)
        : id(i), username(u), displayName(d), bio(b), type(t),
          joinedAt(std::chrono::system_clock::now()), isActive(true
             ) {}

    virtual ~User() = default;

    // Getters
    int getId() const { return id; }
    std::string getUsername() const { return username; }
    std::string getDisplayName() const { return displayName; }
    UserType getType() const { return type; }

    // Friend management
    virtual bool addFriend(int friendId) {
        if (blocked.count(friendId)) return false;
        friends.insert(friendId);
        return true;
    }

    void removeFriend(int friendId) {
        friends.erase(friendId);
    }

    bool isFriend(int userId) const {
        return friends.count(userId) > 0;
    }

    // Follow system
    void follow(int userId) {
        following.insert(userId);
    }

    void addFollower(int userId) {
        followers.insert(userId);
    }

    // Post management
    Post& createPost(ContentData content) {
        posts.emplace_back(posts.size() + 1, id, content);
        return posts.back();
    }

```

```
    const std::vector<Post>& getPosts() const { return posts; }

    // Virtual methods for different user types
    virtual bool canMessage(int userId) const {
        return isFriend(userId);  // Regular users: friends only
    }

    virtual int getMaxPostLength() const { return 500; }
    virtual bool canGoLive() const { return false; }

    // Settings with any type
    template<typename T>
    void setSetting(const std::string& key, T value) {
        settings[key] = value;
    }

    template<typename T>
    std::optional<T> getSetting(const std::string& key) const {
        auto it = settings.find(key);
        if (it != settings.end()) {
            try {
                return std::any_cast<T>(it->second);
            } catch (...) {
                return std::nullopt;
            }
        }
        return std::nullopt;
    }

    // For graph algorithms
    const std::set<int>& getConnections() const { return friends; }

    friend std::ostream& operator<<(std::ostream& os, const User& u
        ) {
        os << "[" << u.id << "] @" << u.username
           << " (" << u.displayName << ") - "
           << u.friends.size() << " friends, "
           << u.followers.size() << " followers";
        return os;
    }
};

class RegularUser : public User {
public:
    RegularUser(int i, std::string u, std::string d, std::string b)
        : User(i, u, d, b, UserType::REGULAR) {}
};

class Influencer : public User {
```

```
private:
    std::string niche;  // fashion, tech, fitness, etc.
    bool isVerified;
    std::vector<std::string> sponsorships;

public:
    Influencer(int i, std::string u, std::string d, std::string b,
               std::string n, bool verified)
        : User(i, u, d, b, UserType::INFLUENCER),
          niche(n), isVerified(verified) {}

    bool canMessage(int userId) const override {
        // Influencers can receive messages from followers too
        return isFriend(userId) || followers.count(userId) > 0;
    }

    int getMaxPostLength() const override { return 2000; }
    bool canGoLive() const override { return isVerified; }

    void addSponsorship(const std::string& brand) {
        sponsorships.push_back(brand);
    }
};

class BusinessUser : public User {
private:
    std::string businessName;
    std::string category;
    std::string website;
    std::map<std::string, std::string> businessHours;
    bool isOpen;

public:
    BusinessUser(int i, std::string u, std::string d, std::string b,
                 std::string name, std::string cat, std::string web)
        : User(i, u, d, b, UserType::BUSINESS),
          businessName(name), category(cat), website(web), isOpen(true) {}

    bool canMessage(int userId) const override {
        return true;  // Businesses accept all messages
    }

    void setHours(const std::string& day, const std::string& hours) {
        businessHours[day] = hours;
    }
```

```

    void toggleOpen() { isOpen = !isOpen; }
};

//


// SOCIAL GRAPH (BFS/DFS algorithms)
//


class SocialGraph {
private:
    std::map<int, std::unique_ptr<User>> users;

public:
    void addUser(std::unique_ptr<User> user) {
        users[user->getId()] = std::move(user);
    }

    User* getUser(int id) {
        auto it = users.find(id);
        return (it != users.end()) ? it->second.get() : nullptr;
    }

    // BFS: Shortest path between users (degrees of separation)
    std::optional<std::vector<int>> findShortestPath(int from, int
       to) {
        if (!getUser(from) || !getUser(to)) return std::nullopt;
        if (from == to) return std::vector<int>{from};

        std::queue<int> q;
        std::map<int, int> parent;  // child -> parent
        std::set<int> visited;

        q.push(from);
        visited.insert(from);

        while (!q.empty()) {
            int current = q.front(); q.pop();

            if (current == to) {
                // Reconstruct path
                std::vector<int> path;
                int node = to;
                while (node != from) {
                    path.push_back(node);
                    node = parent[node];
                }
```

```
                path.push_back(from);
                std::reverse(path.begin(), path.end());
                return path;
            }

            if (User* u = getUser(current)) {
                for (int friendId : u->getConnections()) {
                    if (!visited.count(friendId)) {
                        visited.insert(friendId);
                        parent[friendId] = current;
                        q.push(friendId);
                    }
                }
            }
        }

        return std::nullopt;  // No path found
    }

    // DFS: Find all friends within N degrees
    std::set<int> getNetwork(int userId, int maxDepth) {
        std::set<int> network;
        std::stack<std::pair<int, int>> stack;  // (userId, depth)
        stack.push({userId, 0});

        while (!stack.empty()) {
            auto [current, depth] = stack.top(); stack.pop();

            if (depth > maxDepth || network.count(current)) continue;
            network.insert(current);

            if (User* u = getUser(current)) {
                for (int friendId : u->getConnections()) {
                    stack.push({friendId, depth + 1});
                }
            }
        }

        network.erase(userId);  // Remove self
        return network;
    }

    // Mutual friends
    std::set<int> getMutualFriends(int user1, int user2) {
        User* u1 = getUser(user1);
        User* u2 = getUser(user2);
        if (!u1 || !u2) return {};

```

```
        std::set<int> mutual;
        const auto& friends1 = u1->getConnections();
        const auto& friends2 = u2->getConnections();

        std::set_intersection(friends1.begin(), friends1.end(),
                              friends2.begin(), friends2.end(),
                              std::inserter(mutual, mutual.begin()));

        return mutual;
    }

    // Friend recommendations (friends of friends who aren't friends yet)
    std::vector<std::pair<int, int>> getRecommendations(int userId, int limit = 5) {
        User* u = getUser(userId);
        if (!u) return {};

        std::map<int, int> score;  // potential friend -> mutual count

        for (int friendId : u->getConnections()) {
            User* friend_ = getUser(friendId);
            if (!friend_) continue;

            for (int fof : friend_->getConnections()) {
                if (fof != userId && !u->isFriend(fof)) {
                    score[fof]++;
                }
            }
        }

        // Sort by score
        std::vector<std::pair<int, int>> recs(score.begin(), score.end());
        std::sort(recs.begin(), recs.end(),
                  [](auto& a, auto& b) { return a.second > b.second; });

        if (recs.size() > static_cast<size_t>(limit)) {
            recs.resize(limit);
        }
        return recs;
    }

    void displayStats() const {
        std::cout << "\n=== SOCIAL GRAPH STATS ===\n";
        std::cout << "Total users: " << users.size() << "\n";

```

```
        int totalConnections = 0;
        for (const auto& [id, user] : users) {
            totalConnections += user->getConnections().size();
        }
        std::cout << "Total connections: " << totalConnections << "\n";
        std::cout << "Avg friends per user: "
                  << (users.empty() ? 0 : totalConnections / users.size()) << "\n";
    }
};

//


// NOTIFICATION SYSTEM (Observer pattern)
//


enum class NotificationType { LIKE, COMMENT, FOLLOW, MENTION, MESSAGE };

struct Notification {
    NotificationType type;
    int fromUserId;
    int toUserId;
    std::string message;
    std::chrono::system_clock::time_point createdAt;
    bool isRead;
};

class NotificationManager {
private:
    std::map<int, std::vector<Notification>> userNotifications;

public:
    void sendNotification(NotificationType type, int from, int to,
                          const std::string& msg) {
        Notification notif{type, from, to, msg,
                           std::chrono::system_clock::now(), false};
        userNotifications[to].push_back(notif);
    }

    void notifyLike(int likerId, int postAuthorId, int postId) {
        sendNotification(NotificationType::LIKE, likerId, postAuthorId,
                         "User " + std::to_string(likerId) + " liked your post #"
                         + std::to_string(postId));
```

```
    }

    void notifyFollow(int followerId, int targetId) {
        sendNotification(NotificationType::FOLLOW, followerId,
           targetId,
                         "User " + std::to_string(followerId) + "
                            started following you");
    }

    std::vector<Notification> getUnread(int userId) {
        std::vector<Notification> unread;
        auto it = userNotifications.find(userId);
        if (it != userNotifications.end()) {
            for (const auto& n : it->second) {
                if (!n.isRead) unread.push_back(n);
            }
        }
        return unread;
    }

    void markAllRead(int userId) {
        auto it = userNotifications.find(userId);
        if (it != userNotifications.end()) {
            for (auto& n : it->second) n.isRead = true;
        }
    }
};

#endif
```

Chapter 10

Project 4: Real-time Chat Server with Modern C++

Features

- **OOP:** Abstract connection handler, Message inheritance (Text, Image, File)
- **STL:** deque (message history), unordered_map (sessions), thread_pool
- **Modern C++:** Move semantics, Perfect forwarding, std::jthread (C++20), Atomic operations
- **Design Patterns:** Reactor (event handling), Prototype (message cloning), Connection Pool

Listing 10.1: chat_server.hpp — Complete Project 4

```cpp
#ifndef CHAT_SERVER_HPP
#define CHAT_SERVER_HPP

#include <iostream>
#include <string>
#include <vector>
#include <deque>
#include <map>
#include <unordered_map>
#include <set>
#include <queue>
#include <memory>
#include <thread>
#include <mutex>
#include <shared_mutex>
#include <condition_variable>
#include <atomic>
#include <future>
#include <functional>
#include <chrono>
#include <sstream>
```

```
#include <iomanip>
#include <algorithm>

// C++20 features
#if __cplusplus >= 202002L
#include <semaphore>
#include <latch>
#include <barrier>
#endif

//


// MESSAGE HIERARCHY (Inheritance + Polymorphism)
//


enum class MessageStatus { SENDING, SENT, DELIVERED, READ, FAILED
    };

class Message {
protected:
    int id;
    int senderId;
    int receiverId;
    std::chrono::system_clock::time_point timestamp;
    MessageStatus status;
    bool isDeleted;

public:
    Message(int i, int s, int r)
        : id(i), senderId(s), receiverId(r),
          timestamp(std::chrono::system_clock::now()),
          status(MessageStatus::SENDING), isDeleted(false) {}

    virtual ~Message() = default;

    virtual std::string getContent() const = 0;
    virtual std::string getType() const = 0;
    virtual std::unique_ptr<Message> clone() const = 0;  //
        Prototype pattern

    void markDelivered() { status = MessageStatus::DELIVERED; }
    void markRead() { status = MessageStatus::READ; }

    int getId() const { return id; }
    int getSender() const { return senderId; }
    int getReceiver() const { return receiverId; }
    MessageStatus getStatus() const { return status; }
```

```cpp

    virtual std::string toString() const {
        std::stringstream ss;
        ss << "[" << id << "] " << getType() << " from " << senderId
           << " to " << receiverId << " at " << formatTime(timestamp);
        return ss.str();
    }

protected:
    static std::string formatTime(std::chrono::system_clock::time_point tp) {
        auto time = std::chrono::system_clock::to_time_t(tp);
        std::stringstream ss;
        ss << std::put_time(std::localtime(&time), "%H:%M:%S");
        return ss.str();
    }
};

class TextMessage : public Message {
private:
    std::string text;
    std::vector<std::string> mentions;  // @username references

public:
    TextMessage(int i, int s, int r, std::string t)
        : Message(i, s, r), text(t) {
        parseMentions();
    }

    std::string getContent() const override { return text; }
    std::string getType() const override { return "TEXT"; }

    std::unique_ptr<Message> clone() const override {
        return std::make_unique<TextMessage>(*this);
    }

    const std::vector<std::string>& getMentions() const { return mentions; }

private:
    void parseMentions() {
        size_t pos = 0;
        while ((pos = text.find('@', pos)) != std::string::npos) {
            size_t end = text.find(' ', pos);
            if (end == std::string::npos) end = text.length();
            mentions.push_back(text.substr(pos + 1, end - pos - 1));
```

```
            pos = end;
        }
    }
};

class ImageMessage : public Message {
private:
    std::string imageUrl;
    std::string caption;
    int width, height;
    size_t fileSize;

public:
    ImageMessage(int i, int s, int r, std::string url, std::string cap,
                 int w, int h, size_t size)
        : Message(i, s, r), imageUrl(url), caption(cap),
          width(w), height(h), fileSize(size) {}

    std::string getContent() const override { return "[Image] " + caption; }
    std::string getType() const override { return "IMAGE"; }

    std::unique_ptr<Message> clone() const override {
        return std::make_unique<ImageMessage>(*this);
    }

    std::string getImageUrl() const { return imageUrl; }
};

class FileMessage : public Message {
private:
    std::string fileName;
    std::string fileUrl;
    size_t fileSize;
    std::string mimeType;

public:
    FileMessage(int i, int s, int r, std::string name, std::string url,
                size_t size, std::string mime)
        : Message(i, s, r), fileName(name), fileUrl(url),
          fileSize(size), mimeType(mime) {}

    std::string getContent() const override { return "[File] " + fileName; }
    std::string getType() const override { return "FILE"; }

    std::unique_ptr<Message> clone() const override {
```

```
        return std::make_unique<FileMessage>(*this);
    }

    std::string getFileUrl() const { return fileUrl; }
    size_t getFileSize() const { return fileSize; }
};

//


// THREAD-SAFE MESSAGE HISTORY (STL + Mutexes)
//


class ChatHistory {
private:
    std::deque<std::unique_ptr<Message>> messages;
    mutable std::shared_mutex mutex;  // C++17: shared_mutex for read-heavy
    size_t maxSize;

public:
    explicit ChatHistory(size_t max = 1000) : maxSize(max) {}

    void addMessage(std::unique_ptr<Message> msg) {
        std::unique_lock lock(mutex);
        messages.push_back(std::move(msg));
        if (messages.size() > maxSize) {
            messages.pop_front();
        }
    }

    std::vector<std::unique_ptr<Message>> getRecent(size_t n) const {
        std::shared_lock lock(mutex);
        std::vector<std::unique_ptr<Message>> result;
        auto start = messages.size() > n ? messages.end() - n : messages.begin();
        for (auto it = start; it != messages.end(); ++it) {
            result.push_back((*it)->clone());
        }
        return result;
    }

    std::vector<std::unique_ptr<Message>> search(const std::string& keyword) const {
        std::shared_lock lock(mutex);
        std::vector<std::unique_ptr<Message>> result;
        for (const auto& msg : messages) {
```

```
            if (msg->getContent().find(keyword) != std::string::npos) {
                result.push_back(msg->clone());
            }
        }
        return result;
    }

    size_t size() const {
        std::shared_lock lock(mutex);
        return messages.size();
    }
};

//
// USER SESSION (RAII + Move semantics)
//

class UserSession {
private:
    int userId;
    std::string username;
    std::atomic<bool> isOnline;
    std::atomic<bool> isTyping;
    std::chrono::system_clock::time_point lastActivity;

    mutable std::mutex statusMutex;
    std::condition_variable statusCV;

public:
    UserSession(int id, std::string name)
        : userId(id), username(name), isOnline(true), isTyping(false),
          lastActivity(std::chrono::system_clock::now()) {}

    // Move constructor (efficient transfer)
    UserSession(UserSession&& other) noexcept
        : userId(other.userId), username(std::move(other.username)),
          isOnline(other.isOnline.load()), isTyping(other.isTyping.load()),
          lastActivity(other.lastActivity) {}

    // Disable copy
    UserSession(const UserSession&) = delete;
    UserSession& operator=(const UserSession&) = delete;
```

```

    void setTyping(bool typing) {
        isTyping.store(typing);
        updateActivity();
    }

    bool getTyping() const { return isTyping.load(); }

    void setOnline(bool online) {
        isOnline.store(online);
        if (online) updateActivity();
    }

    bool getOnline() const { return isOnline.load(); }

    void updateActivity() {
        std::lock_guard lock(statusMutex);
        lastActivity = std::chrono::system_clock::now();
    }

    bool isInactive(int minutes) const {
        auto now = std::chrono::system_clock::now();
        auto diff = std::chrono::duration_cast<std::chrono::minutes
           >(now - lastActivity);
        return diff.count() > minutes;
    }

    int getUserId() const { return userId; }
    std::string getUsername() const { return username; }
};

//


// THREAD POOL (Modern C++ implementation)
//


class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;

    std::mutex queueMutex;
    std::condition_variable condition;
    std::atomic<bool> stopFlag;

public:
    explicit ThreadPool(size_t numThreads) : stopFlag(false) {
```

```
        for (size_t i = 0; i < numThreads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock lock(queueMutex);
                        condition.wait(lock, [this] {
                            return stopFlag.load() || !tasks.empty();
                        });

                        if (stopFlag.load() && tasks.empty()) return;

                        task = std::move(tasks.front());
                        tasks.pop();
                    }
                    task();
                }
            });
        }
    }

    ~ThreadPool() {
        stopFlag.store(true);
        condition.notify_all();
        for (auto& worker : workers) {
            if (worker.joinable()) worker.join();
        }
    }

    template<typename F, typename... Args>
    auto enqueue(F&& f, Args&&... args) -> std::future<decltype(f(args...))> {
        using return_type = decltype(f(args...));

        auto task = std::make_shared<std::packaged_task<return_type()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<return_type> result = task->get_future();

        {
            std::unique_lock lock(queueMutex);
            if (stopFlag.load()) throw std::runtime_error("ThreadPool stopped");
            tasks.emplace([task]() { (*task)(); });
```

```
        }

        condition.notify_one();
        return result;
    }
};

//


// CHAT ROOM (Group chat management)
//


class ChatRoom {
private:
    int roomId;
    std::string name;
    std::set<int> memberIds;
    ChatHistory history;
    std::unique_ptr<ThreadPool> messageProcessors;

    mutable std::shared_mutex memberMutex;

public:
    ChatRoom(int id, std::string n)
        : roomId(id), name(n), history(5000),
          messageProcessors(std::make_unique<ThreadPool>(4)) {}

    void addMember(int userId) {
        std::unique_lock lock(memberMutex);
        memberIds.insert(userId);
    }

    void removeMember(int userId) {
        std::unique_lock lock(memberMutex);
        memberIds.erase(userId);
    }

    bool hasMember(int userId) const {
        std::shared_lock lock(memberMutex);
        return memberIds.count(userId) > 0;
    }

    void broadcastMessage(std::unique_ptr<Message> msg) {
        history.addMessage(std::move(msg));

        // Notify all members asynchronously
        std::shared_lock lock(memberMutex);
```

```
        for (int memberId : memberIds) {
            messageProcessors->enqueue([memberId]() {
                // Simulate notification
                std::cout << "Notifying user " << memberId << "\n";
            });
        }
    }

    std::string getName() const { return name; }
    size_t getMemberCount() const {
        std::shared_lock lock(memberMutex);
        return memberIds.size();
    }
};

//


// MAIN CHAT SERVER (Reactor pattern)
//


class ChatServer {
private:
    std::unordered_map<int, std::unique_ptr<UserSession>> sessions;
    std::unordered_map<int, std::unique_ptr<ChatRoom>> rooms;
    std::map<std::pair<int, int>, std::unique_ptr<ChatHistory>>
       privateChats;

    std::unique_ptr<ThreadPool> ioPool;
    std::unique_ptr<ThreadPool> workerPool;

    mutable std::shared_mutex sessionsMutex;
    std::atomic<int> nextRoomId;
    std::atomic<bool> running;

public:
    ChatServer()
        : ioPool(std::make_unique<ThreadPool>(4)),
          workerPool(std::make_unique<ThreadPool>(8)),
          nextRoomId(1), running(true) {}

    // Session management
    void createSession(int userId, std::string username) {
        std::unique_lock lock(sessionsMutex);
        sessions[userId] = std::make_unique<UserSession>(userId,
           username);
        std::cout << "User " << username << " connected\n";
    }
```

```

    void removeSession(int userId) {
        std::unique_lock lock(sessionsMutex);
        sessions.erase(userId);
        std::cout << "User " << userId << " disconnected\n";
    }

    UserSession* getSession(int userId) {
        std::shared_lock lock(sessionsMutex);
        auto it = sessions.find(userId);
        return (it != sessions.end()) ? it->second.get() : nullptr;
    }

    // Room management
    int createRoom(const std::string& name) {
        int id = nextRoomId++;
        rooms[id] = std::make_unique<ChatRoom>(id, name);
        return id;
    }

    void joinRoom(int roomId, int userId) {
        if (auto room = getRoom(roomId)) {
            room->addMember(userId);
        }
    }

    void sendToRoom(int roomId, std::unique_ptr<Message> msg) {
        if (auto room = getRoom(roomId)) {
            room->broadcastMessage(std::move(msg));
        }
    }

    // Private messaging
    void sendPrivateMessage(std::unique_ptr<Message> msg) {
        int sender = msg->getSender();
        int receiver = msg->getReceiver();

        auto key = std::make_pair(std::min(sender, receiver),
                                  std::max(sender, receiver));

        {
            std::unique_lock lock(sessionsMutex);
            if (!privateChats.count(key)) {
                privateChats[key] = std::make_unique<ChatHistory
                    >(10000);
            }
            privateChats[key]->addMessage(std::move(msg));
        }

```

```
        // Deliver to receiver if online
        if (auto session = getSession(receiver)) {
            // Real implementation would push to client's connection
            std::cout << "Delivered message to " << receiver << "\n";
        }
    }

    // Stats
    void displayStats() const {
        std::shared_lock lock(sessionsMutex);
        std::cout << "\n=== SERVER STATS ===\n";
        std::cout << "Active sessions: " << sessions.size() << "\n";
        std::cout << "Active rooms: " << rooms.size() << "\n";
        std::cout << "Private chat channels: " << privateChats.size() << "\n";
    }

private:
    ChatRoom* getRoom(int roomId) {
        auto it = rooms.find(roomId);
        return (it != rooms.end()) ? it->second.get() : nullptr;
    }
};

#endif
```

Part IV

Modern C++17/20/23 Features

Chapter 11

Modern C++ Feature Reference

Listing 11.1: modern_cpp_features.hpp — C++17/20/23

```cpp
#ifndef MODERN_CPP_FEATURES_HPP
#define MODERN_CPP_FEATURES_HPP

#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <optional>
#include <variant>
#include <any>
#include <tuple>
#include <memory>
#include <filesystem>
#include <string_view>
#include <span>
#include <source_location>

// C++20 concepts
#if __cplusplus >= 202002L
#include <concepts>
#include <ranges>
#include <format>
#endif

//


// C++17: STRUCTURED BINDINGS & OPTIONAL
//


void cpp17_features() {
    // Structured bindings
```

```
    std::map<std::string, int> scores = {{"Rahul", 95}, {"Priya", 88}};
    for (const auto& [name, score] : scores) {
        std::cout << name << ": " << score << "\n";
    }

    // std::optional
    std::optional<int> maybeValue;
    if (!maybeValue) std::cout << "No value\n";

    maybeValue = 42;
    std::cout << "Value: " << *maybeValue << "\n";
    std::cout << "Or default: " << maybeValue.value_or(0) << "\n";

    // if with initializer
    if (auto it = scores.find("Rahul"); it != scores.end()) {
        std::cout << "Found: " << it->second << "\n";
    }

    // std::variant (type-safe union)
    std::variant<int, double, std::string> data;
    data = 3.14;
    std::visit([](auto&& arg) {
        using T = std::decay_t<decltype(arg)>;
        if constexpr (std::is_same_v<T, int>) std::cout << "int\n";
        else if constexpr (std::is_same_v<T, double>) std::cout << "double\n";
        else std::cout << "string\n";
    }, data);

    // std::filesystem
    std::filesystem::path p = "/home/user/documents";
    if (std::filesystem::exists(p)) {
        for (const auto& entry : std::filesystem::directory_iterator(p)) {
            std::cout << entry.path() << "\n";
        }
    }

    // std::string_view (no-copy string reference)
    std::string str = "Hello World";
    std::string_view sv(str.c_str(), 5);  // "Hello" without allocation
}

//


// C++20: CONCEPTS & RANGES
```

```
//


#if __cplusplus >= 202002L

// Concept definition
template<typename T>
concept Numeric = std::is_arithmetic_v<T>;

template<typename T>
concept Container = requires(T t) {
    { t.begin() } -> std::same_as<typename T::iterator>;
    { t.end() } -> std::same_as<typename T::iterator>;
    { t.size() } -> std::convertible_to<std::size_t>;
};

// Constrained template
template<Numeric T>
T add(T a, T b) { return a + b; }

template<Container C>
auto sum(const C& container) {
    return std::ranges::fold_left(container, 0, std::plus{});
}

void cpp20_features() {
    // Ranges
    std::vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    auto result = nums
        | std::views::filter([](int n) { return n % 2 == 0; })
        | std::views::transform([](int n) { return n * n; })
        | std::views::take(3);

    for (int n : result) std::cout << n << " ";  // 4 16 36

    // std::format (type-safe printf)
    std::string s = std::format("Hello, {}! You have {} messages.",
                                "Rahul", 5);

    // std::span (non-owning view of contiguous data)
    std::vector<int> data = {1, 2, 3, 4, 5};
    std::span<int> view(data);  // No copy, just pointer + size

    // Designated initializers
    struct Config { int port; std::string host; bool ssl; };
    Config cfg = {.port = 8080, .host = "localhost", .ssl = true};
}

```

```
#endif

//

// C++23 FEATURES (Preview)
//

#if __cplusplus > 202002L

#include <expected>
#include <print>

// std::expected (error handling without exceptions)
std::expected<int, std::string> divide(int a, int b) {
    if (b == 0) return std::unexpected("Division by zero");
    return a / b;
}

void cpp23_features() {
    // std::print (faster than iostream)
    std::print("Hello, {}!\n", "World");

    // std::expected usage
    auto result = divide(10, 2);
    if (result) {
        std::print("Result: {}\n", *result);
    } else {
        std::print("Error: {}\n", result.error());
    }

    // Deducing this (explicit object parameter)
    struct Wrapper {
        int value;

        auto operator()(this auto&& self) {
            return self.value;
        }
    };
}

#endif

//

// PERFECT FORWARDING & MOVE SEMANTICS
```

```
//


template<typename T>
void wrapper(T&& arg) {
    // Perfect forwarding: preserve value category
    foo(std::forward<T>(arg));
}

// Variadic templates with folding
template<typename... Args>
auto sum(Args... args) {
    return (args + ...);  // Fold expression (C++17)
}

// Lambda improvements
void lambda_features() {
    // Generic lambda (C++14)
    auto generic = [](auto x, auto y) { return x + y; };

    // Template lambda (C++20)
    auto templated = []<typename T>(T x, T y) { return x + y; };

    // Stateful lambda with move capture
    auto ptr = std::make_unique<int>(42);
    auto lambda = [p = std::move(ptr)]() { return *p; };
}

#endif
```

Chapter 12

Summary Best Practices

OOP Principles Checklist

1. **Encapsulation:** Data private, access through methods, validation in setters
2. **Inheritance:** "Is-a" relationship, virtual destructors, avoid deep hierarchies
3. **Polymorphism:** Virtual functions, abstract interfaces, runtime dispatch
4. **Abstraction:** Hide complexity, expose intent, use pure interfaces

STL Best Practices

1. **Choose right container:** vector (default), unordered_map (fast lookup), set (sorted unique)
2. **Use algorithms:** Don't write loops, use std::sort, std::find, std::transform
3. **Reserve capacity:** vector::reserve() for known sizes to avoid reallocations
4. **Emplace over insert:** Constructs in-place, avoids copies
5. **Range-based for:** Cleaner syntax, harder to make off-by-one errors

Modern C++ Guidelines

1. **Smart pointers:** unique_ptr (ownership), shared_ptr (shared), weak_ptr (break cycles)
2. **RAII:** Resources acquired in constructor, released in destructor
3. **Move semantics:** Use std::move for rvalues, enable move constructors
4. **Const correctness:** Mark methods const, use const references
5. **Auto:** Use for complex types, iterators, lambdas
6. **nullptr:** Never use NULL or 0 for pointers